1. Problem Statement, Objectives, and Assumptions

Problem Statement

Managing bank account balances and tracking day-to-day deposits or withdrawals manually can lead to errors. There is a need for a command-line simulation tool that allows users to initialize an account balance, execute multiple deposit or withdrawal transactions with proper validation rules, and view a detailed statement alongside the final balance.

Objectives

- Initialize an account with a user-defined opening balance.
- Process continuous user transactions (deposits and withdrawals) until an exit command (0) is entered.
- Validate user inputs to prevent negative amounts, invalid transaction types, and overdrafts (withdrawals exceeding the current balance).
- Maintain a chronological log of all successful transactions.
- Display an itemised transaction summary and the final account balance.

Assumptions

The user interacts via a command-line interface and provides numerical values when prompted for monetary amounts.

-
-

Transactions occur in a single-user, session-based environment without persistent database storage.

-
-

Currency is denoted in Indian Rupees (₹) as per the implementation.

-

2. Algorithm, Flowchart, and Pseudocode

Algorithm

1.

Prompt the user for an opening balance and convert it to a floating-point number.

2.
3.

Initialize an empty list called transactions.

4.
5.

Start a loop to accept transactions:

6.
   -

Read the transaction type (D for deposit, W for withdrawal, O to finish).

   -
   -

If input is O, break out of the loop.

   -
   -

If input is neither D nor W, display an error message and continue to the next iteration.

   -

- Prompt for the transaction amount.
-
- If the amount is less than or equal to zero, display an error message.
-
- Else if the type is D, add the amount to the balance and record the deposit.
-
- Else if the type is W and the amount is less than or equal to the balance, subtract the amount from the balance and record the withdrawal.
-
- Otherwise, reject the withdrawal due to insufficient balance.
-

7. Print the final transaction history and the remaining balance.
8. **Flowchart Representation**

Pseudocode

3. Python Source Code and Test Cases

Python Source Code

```
          float input "Opening balance: "

while True
          input "Enter D=deposit, W=withdraw, O=finish: "
   if           "O"
      break

   if          "D" and          "W"
             "Invalid transaction type."
      continue

             float input "Enter amount: "
   if               0
             "Amount must be positive."
   elif           "D"

                              "Deposit"
             "Deposit successful."
   elif

                              "Withdrawal"
             "Withdrawal successful."
   else
             "Rejected: insufficient balance."
```

```
        "\nTransactions:" for                in
                "₹"  round          2
        "Final balance: ₹"  round          2
```

(Source code structure adapted from).

Test Cases & Sample Output

Test Case ID	Scenario Description	User Inputs	Expected Output / Behavior
TC-01	Standard Deposit & Withdrawal	Opening: 5000	

D, 1500

W, 2000

0 | Successful deposit and withdrawal; Final balance: ₹4500.0. | |
TC-02 | Insufficient Balance Rejection | Opening: 1000

W, 1500

0 | Rejection message printed; Final balance remains ₹1000.0. | |
TC-03 | Invalid Transaction Type | Opening: 2000

X

0 | "Invalid transaction type" message; program loops to exit. |

Simulated Terminal Output (for TC-01)

4. Conclusion

The implemented ATM transaction script effectively demonstrates core programming concepts such as loops, conditional statements, input validation, and list manipulation. It handles boundary conditions gracefully—such as preventing negative inputs and avoiding overdrafts—making it a reliable foundational tool for basic financial tracking simulations.